CORE JAVA – IMPORTANT YES/NO QUESTIONS WITH REASONS

1. Can Java support multiple inheritance through classes? Answer: No. Reason: To avoid ambiguity (Diamond Problem).

2. Can Java support multiple inheritance through interfaces? Answer: Yes. Reason: A class can implement multiple interfaces.

3. Can constructors be inherited? Answer: No. Reason: Constructors belong to their own class.

4. Can constructors be overridden? Answer: No. Reason: Constructors are not inherited.

5. Can static methods be overridden? Answer: No. Reason: Static methods are hidden, not overridden.

6. Can final methods be overridden? Answer: No. Reason: The final keyword prevents overriding.

7. Can private methods be overridden? Answer: No. Reason: Private methods are not inherited.

8. Can a final class be inherited? Answer: No. Reason: Final classes cannot have subclasses.

9. Can we create an object of an abstract class? Answer: No. Reason: Abstract classes are incomplete.

10. Can an abstract class have constructors? Answer: Yes. Reason: To initialize inherited data.

11. Can an abstract class have non-abstract methods? Answer: Yes. Reason: It can contain both abstract and concrete methods.

12. Can an abstract class have static methods? Answer: Yes. Reason: Static methods belong to the class.

13. Can an interface have constructors? Answer: No. Reason: Interfaces cannot be instantiated.

14. Can an interface have variables? Answer: Yes. Reason: They are public, static, and final by default.

15. Can an interface have method implementations? Answer: Yes. Reason: Using default and static methods.

16. Can a class implement multiple interfaces? Answer: Yes. Reason: Java supports multiple interface implementation.

17. Can an interface extend another interface? Answer: Yes. Reason: Interfaces support inheritance.

18. Is method overloading compile-time polymorphism? Answer: Yes. Reason: The compiler selects the method.

19. Is method overriding runtime polymorphism? Answer: Yes. Reason: The JVM selects the method at runtime.

20. Is upcasting required for runtime polymorphism? Answer: Yes. Reason: A parent reference refers to a child object.

21. Can runtime polymorphism occur without inheritance? Answer: No. Reason: It depends on method overriding.

22. Can static methods participate in runtime polymorphism? Answer: No. Reason: They are resolved at compile time.

23. Can final methods participate in runtime polymorphism? Answer: No. Reason: They cannot be overridden.

24. Can private methods participate in runtime polymorphism? Answer: No. Reason: They cannot be overridden.

25. Is String immutable? Answer: Yes. Reason: Its value cannot be changed after creation.

26. Is String a class? Answer: Yes. Reason: It is a predefined Java class.

27. Can String objects be modified after creation? Answer: No. Reason: Strings are immutable.

28. Is StringBuffer mutable? Answer: Yes. Reason: Its content can be changed.

29. Is StringBuilder mutable? Answer: Yes. Reason: Its content can be changed.

30. Is StringBuffer thread-safe? Answer: Yes. Reason: Its methods are synchronized.

31. Is StringBuilder thread-safe? Answer: No. Reason: Its methods are not synchronized.

32. Can we compare Strings using == ? Answer: Yes. Reason: It compares references, not content.

33. Should Strings be compared using equals()? Answer: Yes. Reason: It compares actual content.

34. Is Exception a class? Answer: Yes. Reason: It is the parent of most exceptions.

35. Is Error a subclass of Throwable? Answer: Yes. Reason: Both Error and Exception inherit Throwable.

36. Can a try block exist without a catch block? Answer: Yes. Reason: If a finally block is present.

37. Can a catch block exist without a try block? Answer: No. Reason: Catch must follow try.

38. Can a finally block exist without a try block? Answer: No. Reason: Finally is associated with try.

39. Is finally always executed? Answer: Yes. Reason: Except when the JVM terminates abnormally.

40. Can we have multiple catch blocks? Answer: Yes. Reason: Different exceptions can be handled separately.

41. Can we write code after throw? Answer: No. Reason: Control immediately transfers to exception handling.

42. Can we throw our own exceptions? Answer: Yes. Reason: Using user-defined exceptions.

43. Does ArrayList allow duplicates? Answer: Yes. Reason: Duplicate elements are permitted.

44. Does ArrayList maintain insertion order? Answer: Yes. Reason: Elements are stored in insertion order.

45. Does HashSet allow duplicates? Answer: No. Reason: Sets store unique values.

46. Does HashSet maintain insertion order? Answer: No. Reason: Order is not guaranteed.

47. Does LinkedHashSet maintain insertion order? Answer: Yes. Reason: It preserves insertion order.

48. Does TreeSet store data in sorted order? Answer: Yes. Reason: Elements are automatically sorted.

49. Can HashMap contain one null key? Answer: Yes. Reason: Only one null key is allowed.

50. Can HashMap contain multiple null values? Answer: Yes. Reason: Multiple null values are allowed.

51. Does HashMap maintain insertion order? Answer: No. Reason: Order is not guaranteed.

52. Does LinkedHashMap maintain insertion order? Answer: Yes. Reason: It preserves insertion order.

53. Does TreeMap store keys in sorted order? Answer: Yes. Reason: Keys are automatically sorted.

54. Can an ArrayList store different types of objects? Answer: Yes. Reason: When generics are not used.

55. Can an interface be instantiated? Answer: No. Reason: Interfaces cannot have objects directly.

56. Can an abstract class be instantiated? Answer: No. Reason: Abstract classes are incomplete.

57. Can a class be both abstract and final? Answer: No. Reason: Abstract requires inheritance, final prevents inheritance.

58. Can main() be overloaded? Answer: Yes. Reason: Java allows method overloading.

59. Can main() be overridden? Answer: No. Reason: Static methods cannot be overridden.

60. Can static variables be accessed without creating an object? Answer: Yes. Reason: Static members belong to the class.